UPN, PASIÓN POR TRANSFORMAR VIDAS

ANALISIS DE ALGORITMOS Y ESTRATEGIAS DE PROGRAMACION

SESIÓN 9: BackTracking

Ing. Mitchell Blancas

LOGRO DE LA SESIÓN



Al término de la sesión el estudiante aplica Backtracking en la solución de problemas propuestos y los representa a través de algoritmos.

¡Pregunta!

¿En que casos se aplica backtracking?



Búsqueda con retroceso o Backtracking



Es una técnica que consiste en recorrer sistemáticamente todos los posibles caminos. Cuando un camino no conduce a la solución, se retrocede al paso anterior para buscar otro camino.

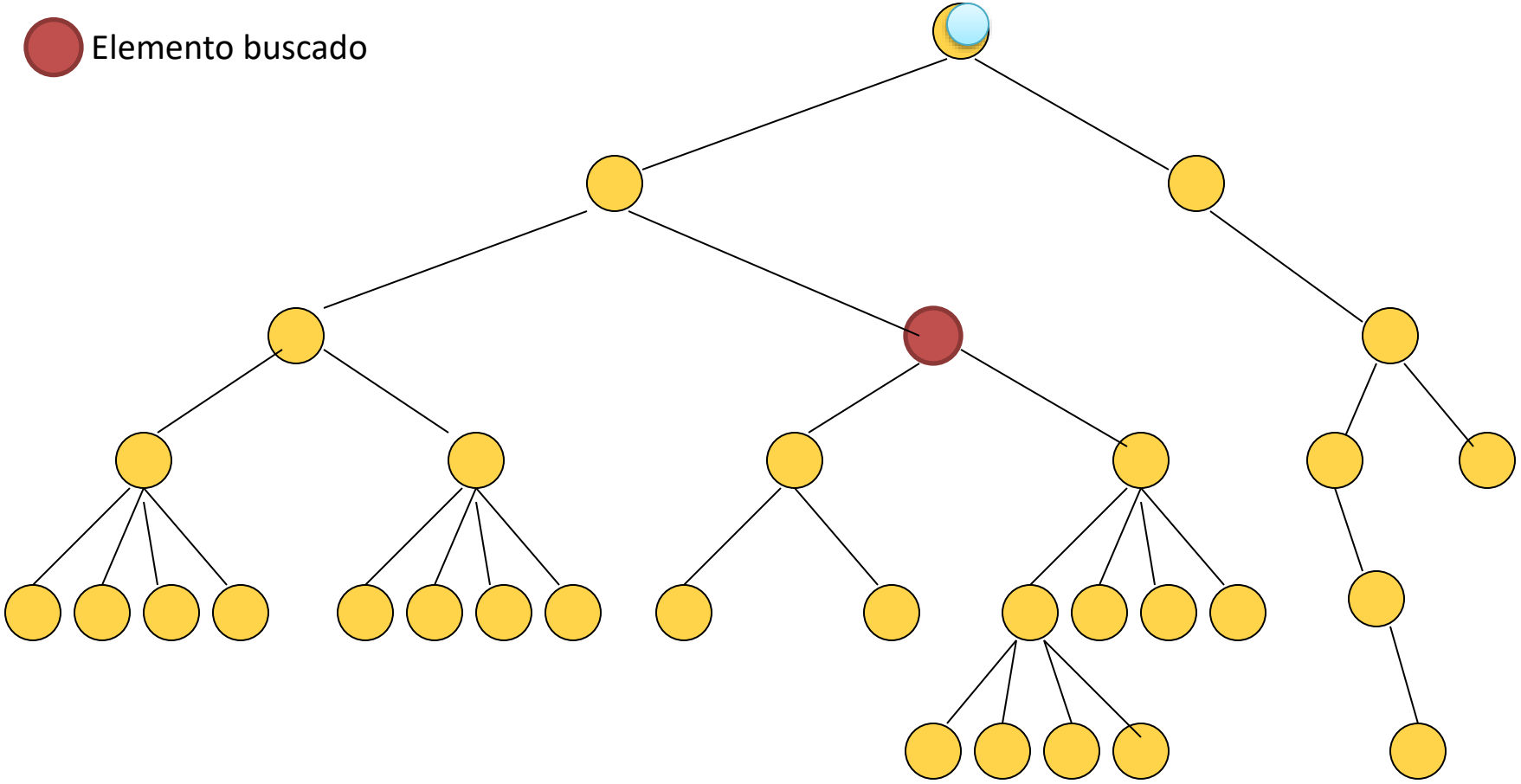
Si existe solución, es seguro que se encuentra. El problema viene a ser el tiempo de proceso.

Es deseable que solo recorra los caminos que conduzcan a la solución y no todos. Es posible hacerlo cuando estando en un punto nos damos cuenta que si seguimos avanzando no encontraremos ninguna solución.

Es un técnica netamente recursiva, aunque puede implementarse iterativamente.

El backtracking básicamente es un recorrido hasta el fondo de un árbol, aunque el árbol es implícito.

A este tipo de búsqueda se le conoce como Búsqueda Primero en Profundidad ó DFS (*Depth First Search*)



Características del árbol



El algoritmo de “Vuelta atrás” hace un recorrido en profundidad en un árbol.
El árbol se construye en memoria por eso se dice que es implícito.
El recorrido en profundidad regresa sobre sus pasos (retrocede) cada vez que encuentra un camino por el que no debe o no puede continuar.

Dimensiones del árbol



La Altura

Hay k posibles decisiones que debemos tomar para llegar a una solución.

La Anchura

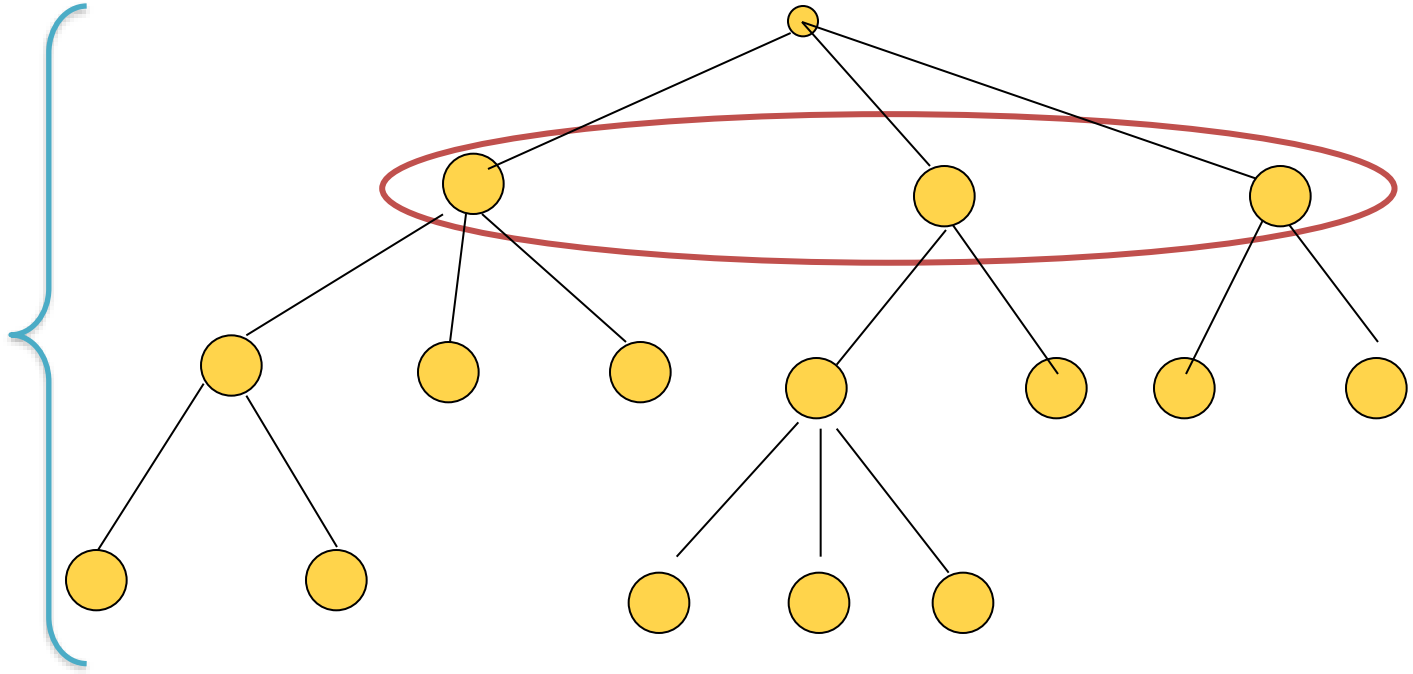
Cada decisión debe tomarse de un conjunto de alternativas distintas (dominio).

Dimensiones



Anchura: número de alternativas (candidatos)

Altura (profundidad): La cantidad de decisiones que debemos tomar



Cuando queramos aplicar backtracking debemos primero imaginarnos como será el árbol que vamos a recorrer!

Esquema general:



Dado que en todo algoritmo de ***backtracking*** hay que recorrer un árbol y en cada paso se necesita encontrar todas las alternativas posibles y luego recorrerlas una a una, es que podemos pensar en el esquema siguiente:

1. ***Si se llego a la máxima profundidad del árbol, entonces mostrar solución (caso base)***
2. ***Sino entonces***
   2.1 Construir cada candidato
   2.2 Para cada candidato añadirlo a la solución y llamar recursivamente a la función con el nivel siguiente de profundidad.

Lo que cambia en cada problema es:

1) La forma de generar cada candidato
2) Los tipos de datos
3) El número de parámetros de la funciones
4) Añadirle condiciones adicionales propias del problema
5) Añadirle condiciones adicionales para reducir el árbol (podar)



Selección de objetos

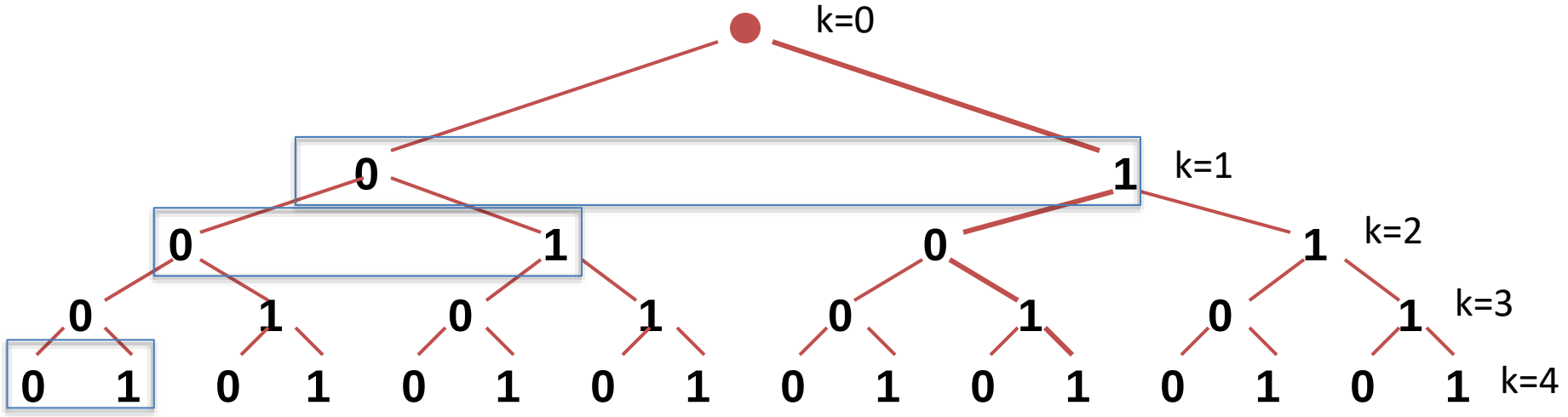
Se tienen n objetos, cada uno puede o no ser seleccionado, ¿Cuáles son todas las formas posibles de tomarlos? ¿De cuántas maneras los podemos escoger?



Supongamos que tenemos 4 objetos …

Primero hay que pensar como será nuestro árbol:

- En cada nivel podemos decidir si elegir o no el artículo.
- La cantidad de decisiones que se deben tomar depende de la cantidad de artículos.
- Dicha cantidad de decisiones es la profundidad (altura) del árbol.



Por ejemplo, el camino **1011** significa que se tomaron el primer, tercer y cuarto artículos y no se tomo el segundo.

Código

```
# include <iostream>
using namespace std;
# define MAX 20
void BT(int, int, bool [ ]);
void imprimir(int, bool[ ]);
int construyeCandidatos(bool [ ]);
void main(void)
{  int i, n;
   bool decision[MAX];
   cout<<"Nro de elementos:";
   cin>>n;

  // marcamos objetos como no escogidos
   for(i=0; i<n; i++)
            decision[i]=false;
   BT(0, n, decision);
}

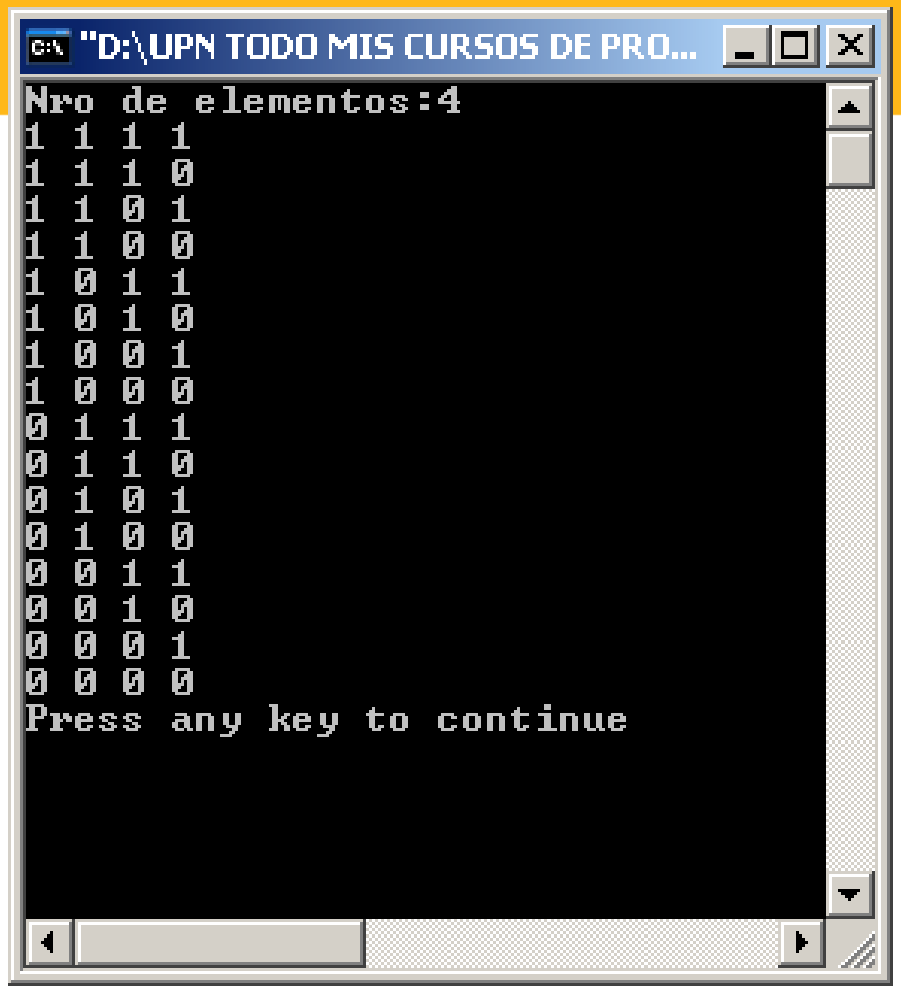
void imprimir(int n, bool decision[ ]) {
   for(int i=0; i<n; i++)
            cout<<decision[i]<<" ";
   cout<<endl;
}
```

```
void BT (int k, int n, bool decision[ ])
{
   bool cand[MAX];

   if (k==n)
        imprimir(n, decision);
   else {
      int ncand=construyeCandidatos(cand);
      for(int i=0; i<ncand; i++) {
           decision[k]= cand[i];
           BT(k+1, n, decision);
      }
   }
}

int construyeCandidatos(bool cand[]) {
    cand[0]=true;
    cand[1]=false;
    return 2;
}
```

1. *Si se llego a la máxima profundidad del árbol, entonces mostrar solución (caso base)*
2. *Sino entonces*
   2.1 Construir cada candidato
   2.2 Para cada candidato añadirlo a la solución y llamar recursivamente a la función con el nivel siguiente de profundidad.


"D:\UPN TODO MIS CURSOS DE PRO...
Nro de elementos:4
1 1 1 1
1 1 1 0
1 1 0 1
1 1 0 0
1 0 1 1
1 0 1 0
1 0 0 1
1 0 0 0
0 1 1 1
0 1 1 0
0 1 0 1
0 1 0 0
0 0 1 1
0 0 1 0
0 0 0 1
0 0 0 0
Press any key to continue

Cálculo de la complejidad



Dado que en cada llamada recursiva se invoca 2 veces a la misma función y como hay **n** invocaciones por cada llamada recursiva, su complejidad es:

$$\Theta(2^n)$$

Note que al tener una complejidad exponencial, el ***backtraking*** no es una técnica muy eficiente, pero su ventaja es que siempre encontrará la solución si es que existe y si tuviéramos todo el tiempo del mundo.

En este caso el análisis de complejidad nos permite darnos una idea de que tan rápido podemos obtener la solución, o si es factible encontrarla, en caso de no serlo debemos echar mano de otras técnicas.

Podemos elegir no recorrer todos los nodos del árbol si sabemos que no nos llevará a la solución buscada. Para esto usamos **restricciones** que pueden ser dadas en el enunciado de manera explícita o implícita.

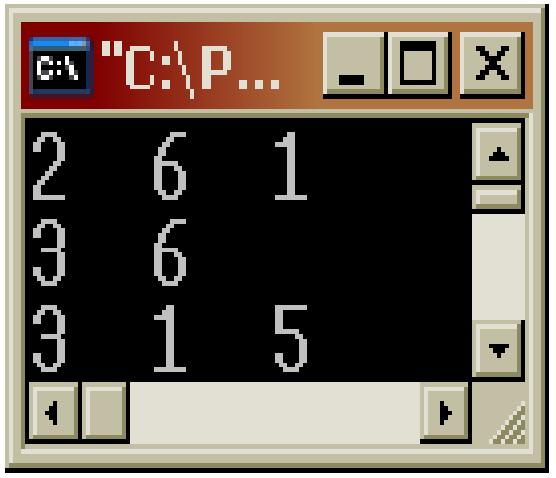
Existen técnicas para **"*podar*"** el árbol de búsqueda, tales como **poda alfa beta**, **algoritmo A***, o algún otro tipo de **heurística**.

Subconjuntos que suman valor objetivo

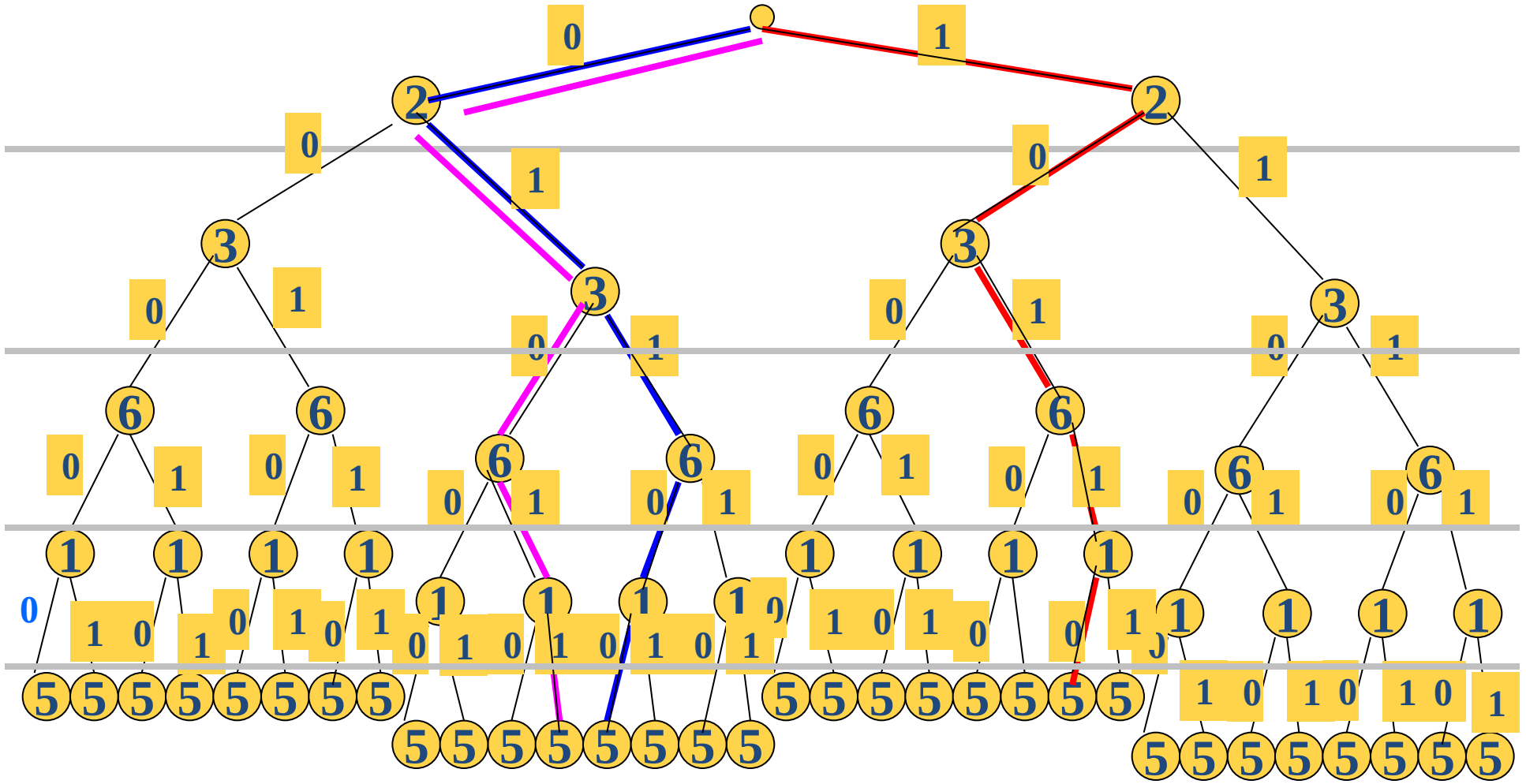
Dado un conjunto de números enteros encontrar todos los subconjuntos cuya suma sea exactamente un valor objetivo.

Por ejemplo, sea el conjunto {2, 3, 6, 1, 5}, y si pedimos los subconjuntos que sumen 9, éstos serán:



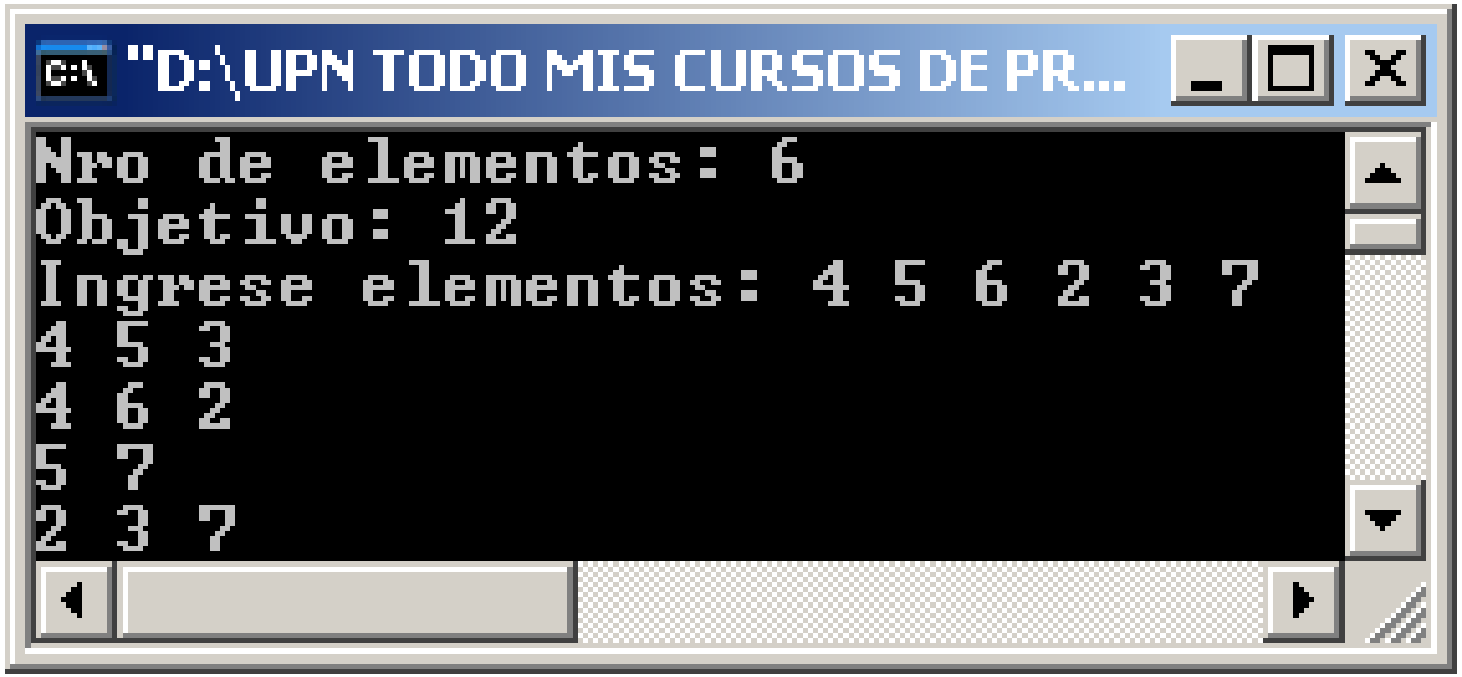


Pensemos en nuestro árbol… si tenemos los elementos **{2, 3, 6, 1, 5}** entonces tendrá **5** niveles y en cada nivel debo decidir si tomar o no el elemento.



```
#include <iostream>
using namespace std;
#define LIM 20
int suma(int);
int buscaCandidatos(bool []);
void bt(int);
void impSol(int);
int n;                    // nro de elementos
int objetivo;             // suma objetivo
int x[LIM];               // elementos
bool seleccion[LIM];      // objetos elegidos
int main()
{  int i;
   cout<<"Nro de elementos: ";  cin>>n;
   cout<<"Objetivo: ";          cin>>objetivo;
   // marco todos como no seleccionados
   for(i=0; i<n; i++) seleccion[i]=false;
   cout<<"Ingrese elementos: ";
   for(i=0; i<n; i++)  cin>>x[i];
   bt(0);
}
void impSol(int k)
{  for(int j=0; j<=k; j++)
       if( seleccion[j]==1 )
          cout<<x[j]<<" ";
   cout<<endl;
}
```

```
void bt(int k)
{   bool cand[LIM];
    if (k==n) return;  // condición base
   int ncand = buscaCandidatos(cand);
   for(int i=0; i<ncand; i++)
   {    seleccion[k] = cand[i];
        int s=suma(k);
        if( s==objetivo )  impSol(k);
        else if( s<objetivo )  bt(k+1);
      }
}
int buscaCandidatos(bool cand[ ])
{    cand[0]=true;
     cand[1]=false;
     return 2;
}
int suma(int k)
{     int sum=0;
      for(int j=0; j<=k; j++)
            if( seleccion[j]==1)  sum += x[j];
      return sum;
}
```


"D:\UPN TODO MIS CURSOS DE PR...
Nro de elementos: 6
Objetivo: 12
Ingrese elementos: 4 5 6 2 3 7
4 5 3
4 6 2
5 7
2 3 7



Variaciones

Encuentre las **variaciones** de n elementos tomados de r en r.

Las variaciones son las distintas formas en que pueden tomarse r elementos de un total de n.

¿Cuántas son?

$$V_n^r = \frac{n!}{(n-r)!}$$

$$\frac{4!}{(4-3)!} = 4x3x2 = 24 \text{ formas distintas}$$

¿Cuáles son?

abc	bac	cab	dab
abd	bad	cad	dac
acb	bca	cba	dba
acd	bcd	cbd	dbc
adb	bda	cda	dca
adc	bdc	cdb	dcb

UNIVERSIDAD PRIVADA DEL NORTE

```
# include <iostream>
# define MAX 30
using namespace std;
char x[MAX], rpta[MAX];
int n, r;
void imp( ) {
   for(int i=0; i<r; i++)
            cout<<rpta[i];
   cout<<endl;
}
int construyeCandidatos(int k, char cand[ ]) {
   int i, j, ncand=0;
   bool encontrado;
   for(i=0; i<n; i++) {
            encontrado=false;
            for(j=0; j<k; j++)
                  if(x[i]==rpta[j])
                        encontrado=true;
            if(!encontrado) {
                        cand[ncand]=x[i];
                        ncand++;
            }
   }
   return ncand;
}
```

```
void bt(int k) {
   char cand[MAX];
   if(k==r) imp();
   else {
      int ncand=construyeCandidatos(k, cand);
      for(int i=0; i<ncand; i++) {
            rpta[k]=cand[i];
            bt(k+1);
      }
   }
}
int main() {
    cout<<"Ingrese Elementos:";
    cin>>x;
    cout<<"n="; cin>>n;
    cout<<"r="; cin>>r;
    cout<<endl<<"Las variaciones son:"<<endl;
    bt(0);
}
```

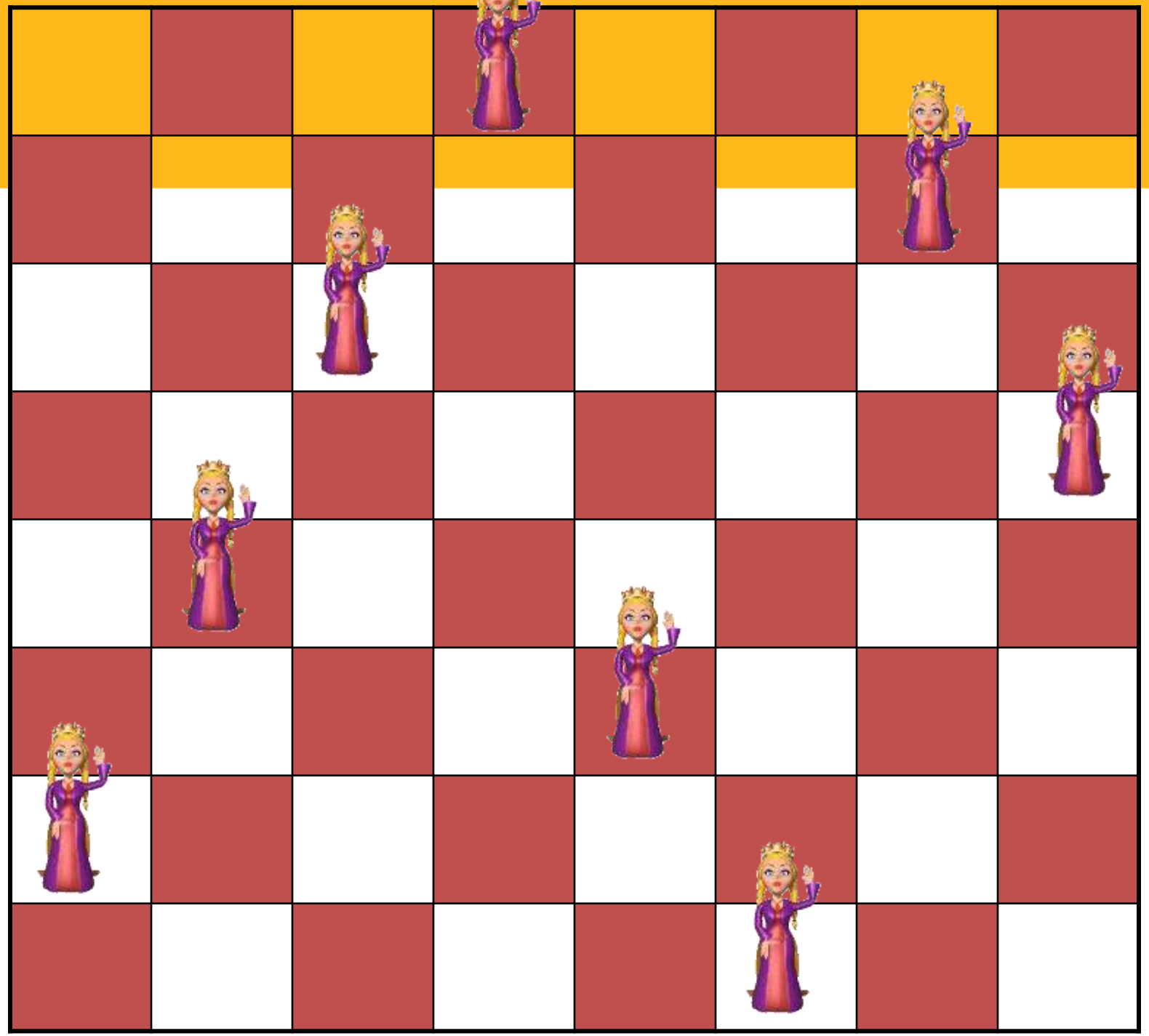
EL PROBLEMA DE LAS N REINAS

En un tablero de ajedrez coloque 8 reinas sin que se maten.



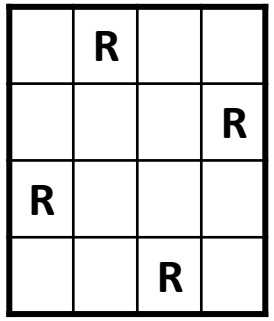


8
7
6
5
4
3
2
1

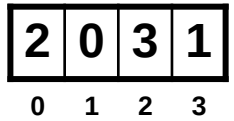


Ejemplo:

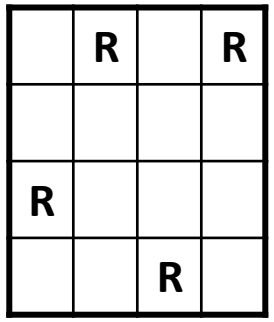
Vectores para el Problema 4 reinas

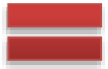


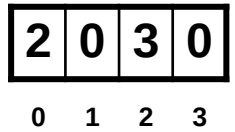






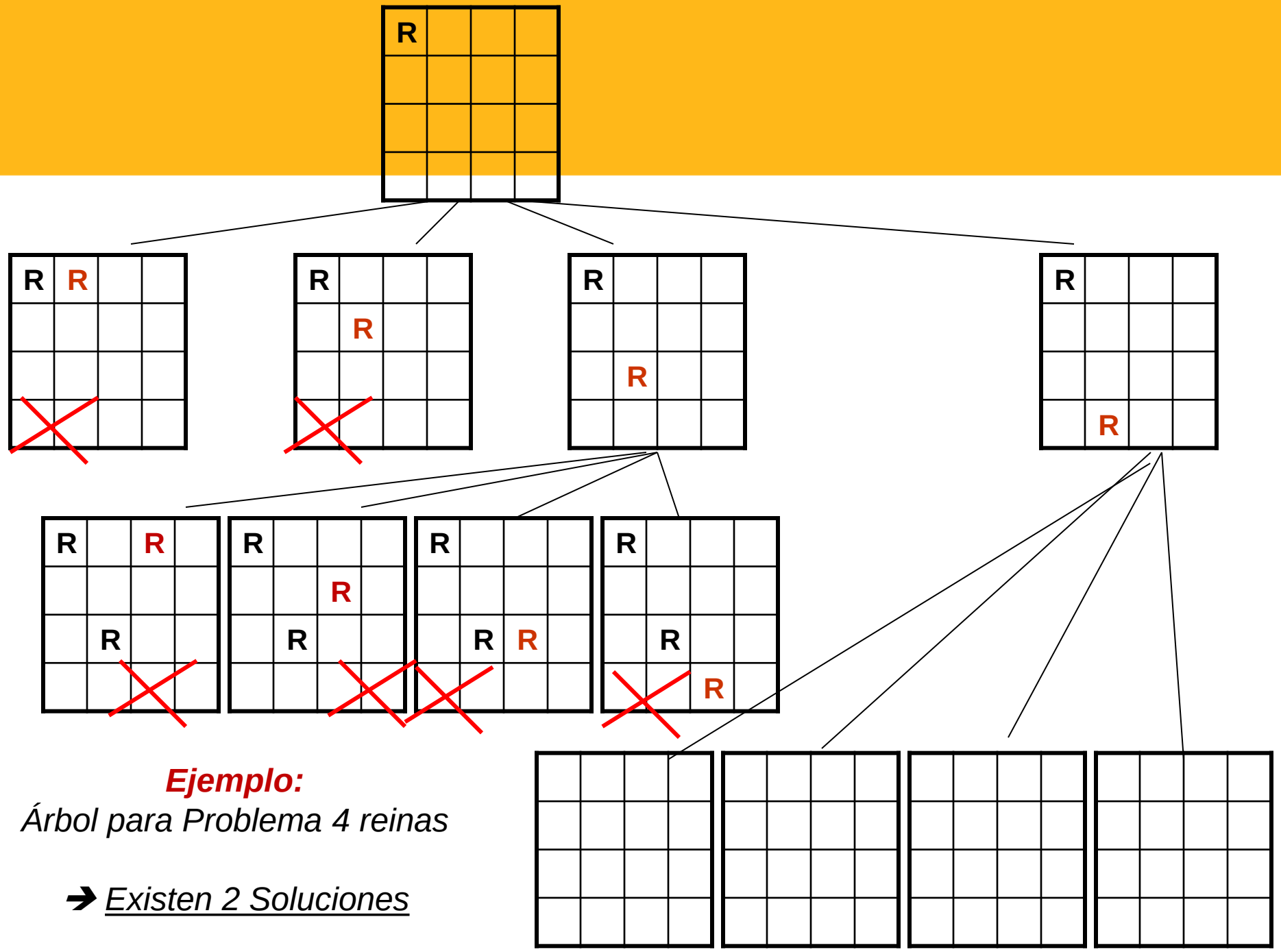








Como no puede haber mas de una reina en una misma columna, entonces podemos restringir el número de candidatos desde un principio, usando un vector donde cada posición i, indica la columna en que se encuentra la reina, y el valor indica la fila en que se encuentra la reina.



Ejemplo:

Árbol para Problema 4 reinas

➔ *<u>Existen 2 Soluciones</u>*

EL PROBLEMA DE LAS N REINAS



```
# include <iostream>
using namespace std;
#define MAXCANDIDATOS 100
#define NMAX  20              // máximo tamaño del
tablero
int nroSol,n;                                 // cuenta
soluciones
int solucion[NMAX];
void imprimeSolucion() {
            for(int i=0; i<n; i++) {
                        for(int j=0; j<n; j++)
                                    if( i==solucion[j] )
                                                cout<<'R'<<' ';
                                    else
                                                cout<<'-'<<' ';
                        cout<<endl;
            }
            cout<<endl;
            nroSol ++;
}
```

EL PROBLEMA DE LAS N REINAS



```
int construyeCandidatos(int k, int candi[]) {
        int ncandidatos = 0;
        for (int i=0; i<n; i++) {
                bool band = true;
                for (int j=0; j<k; j++) {
                  // restriccion de diagonal y de fila
                  if ( abs(k-j) == abs(i-solucion[j]) || i == solucion[j] )
                     band = false;
                }
                if (band) {
                        candi[ncandidatos] =
                        ncandidatos++;
                }
        }
        return ncandidatos;
}
```

```
int main(){
        cout<<"Medida del tablero: ";
        cin>>n;
        nroSol = 0;
        BT(0);
        cout<<"Num. Soluciones="<<
nroSol;
        return 0;
}
```

TAREA:

Agregar función BT()


Kahoot!

PRÓXIMAMENTE...

Programación dinámica



¡Gracias por su atención!

